

Gamification, Social Cohort, Notification and Retention Experimentation Engine

1 Problem Boundaries and Assumptions

- **Supported Rewards:** The primary reward mechanism is Experience Points (XP), bounded by an algebraic decay model that diminishes returns on repeated lessons to prevent reward farming.
- **Streak Model:** Daily streaks are strictly calculated using the learner’s localized IANA timezone to determine day boundaries, incorporating a “freeze” inventory that protects users from losing their streak during inactivity.
- **Cohorts and Leaderboards:** Learners are deterministically assigned to scoped cohorts, which drive participation in sub-millisecond, Redis-backed leaderboards separated from global rankings.
- **Notification Channels and Constraints:** Interventions are delivered via push notifications, but are strictly capped at 5 messages per 24-hour period per learner, and must respect localized quiet hours to prevent notification fatigue.
- **Synthetic-Population Size:** The system assumes a high-volume synthetic user base specifically designed to stress-test peak-load ingestion, P99 latency, and Kafka partition concurrency without database locking bottlenecks.
- **System Exclusions:** The platform handles scripted point farming and impossible timing exploits asynchronously. A dedicated Trust/abuse detection consumer evaluates the event stream independently and emits retroactive compensating events (such as negative XP awards or account flags) to correct the state of abusive accounts without blocking real-time gamification ingestion.
- **Interface Boundaries (Dashboard Scope):** The dashboard is strictly an internal, read-only debugging tool to inspect raw event logs, cohort states, and abuse flags. It is not a learner-facing product and explicitly excludes UI styling, push updates, or interactive features to keep focus entirely on M1–M8 backend milestones.

2 Literature and Landscape Analysis

2.1 Literature Analysis

1. Does gamification work?

- A literature review of 24 empirical studies examining gamification across contexts such as education, health and exercise, online services etc.
- The goal of the paper is to evaluate the effectiveness of gamification across different contexts, and they found that in the majority of cases, it positively affected motivation and behavior.
- However, the effects depend on the context and users, implying that gamification should be tailored and experimentally evaluated rather than universally applied.

2. The Gamification of Learning: A Meta-analysis

- A meta-analysis of empirical studies on **gamification in learning**, examining cognitive, motivational, and behavioral outcomes.
- It separates the effects of gamification into three categories:
 - **Cognitive outcomes:** Whether gamification actually improves learning-related performance, such as knowledge acquisition, understanding, or academic performance.
 - **Motivational outcomes:** Whether learners become more motivated, interested, or engaged with the learning activity.
 - **Behavioral outcomes:** Whether learners actually change their behavior, such as participating more, spending more time on tasks, or completing activities.
- They found **small but significant positive effects of gamification on all three outcomes:** cognitive learning, motivation, and learning-related behavior. The cognitive effect was the most stable when looking only at studies with stronger methodology.
- However, the **motivational and behavioral effects were not robust** in the higher-quality studies, and results varied considerably depending on how gamification was implemented. This implies that gamification alone is not a guaranteed solution; the specific game elements and social setup need to be designed and tested for the intended outcome.

3. Streaks to Success? The Effects of Highlighting Streaks on Student Effort and Learning

- The paper studies **whether highlighting learning streaks can increase students' effort and actual learning** on an online math platform, using a randomized experiment with about **60,000 students**. Students received either streak messages, personalized reminders, generic reminders, or no intervention.
- The goal was to test whether **streaks and reminders change students' platform use and learning**. They found that **both streak messages and personalized reminders increased platform use**, while streak messages also produced a **significant improvement in math achievement compared with the control group**.

- However, the learning result comes from only **about 1,500 students who completed the endline test**, and the streak treatment was **not significantly different from the other treatment groups** on learning. This implies that **streaks provide stronger evidence for increasing effort/use than for improving learning itself**.

4. Increasing Students' Engagement to Reminder Emails Through Multi-Armed Bandits.

- The paper studies **using multi-armed bandits to adaptively choose between different reminder-email subject lines** for students in a first-year CS course, with **email open rate** as the engagement measure.
- The goal is to test whether **adaptive bandit allocation can improve student engagement with reminder emails compared with uniform random assignment**. They found that bandit-based allocation could **shift more students toward subject lines that appeared more effective**, but the results also showed that the bandit could favor an arm even when there was **no statistically significant difference between the arms**.
- The main catch is the **exploration–exploitation trade-off**: a bandit may exploit an apparently good option too early and therefore collect insufficient evidence about the alternatives. The study also used **email opens rather than learning outcomes**, and was conducted in a single CS1 course.

5. Effect of Personalized Email-Based Reminders on Participants' Timeliness in an Online Education Program: Randomized Controlled Trial

- The paper studies **whether personalized email reminders can improve the timeliness of adult learners' coursework**, comparing personalized reminders with general reminders in an online education program. The study involved **39 adult professionals from Ethiopia, Rwanda, and Kenya**.
- The goal was to test whether **adding information about a learner's current progress to regular reminders would help them stay on schedule**. They found that personalized reminders increased the estimated probability of being on time by **14 percentage points** compared with general reminders.
- However, **both groups received reminders**, the sample was only 39 learners, and all participants were adult professionals from three East African countries. The study also measured **timeliness rather than learning itself**. This implies that personalization can improve adherence, but its effect on learning and in other learner populations still needs to be tested.

2.2 Landscape Analysis

1. Duolingo:

- A proprietary language-learning app that uses gamification as part of its learner experience, rather than offering a gamification service for other platforms to integrate.

- **Features it offers:** Weekly leagues, streaks, reminders, social quests, XP, and opt-out controls. Duolingo also describes testing features through internal A/B experiments.
- Duolingo is a useful example of learner-facing features and experimentation, but it is a closed product, not a reusable backend service. Its public materials don't reveal its event model, replay and recovery mechanisms, detailed abuse-detection methods, or notification infrastructure.

2. Funifier:

- A commercial gamification platform that can be integrated with business and learning applications.
- **Features it offers:** A REST API, SDKs, triggers, integrations, widgets, configurable game strategies, and features such as points, challenges, and leaderboards. Its case studies include learning and training uses.
- Funifier appears to cover configurable gamification and integration, but its public materials don't establish your deeper technical goals: a versioned event model, deterministic replay and recovery, detailed abuse detection, or controlled experiments with learning and fatigue measures.

3. FGPE Gamification Service:

- An open-source GraphQL service for adding gamification to programming education. It uses GEdIL, a format for describing gamification rules and learning activities.
- **Features it offers:** It connects gamified challenges to automatically assessed programming activities. Its public repository includes the service source code and Docker-based setup.
- FGPE is focused on programming courses and challenge-based learning. The materials reviewed don't establish a general event-processing system, cohort and global leaderboards, multi-channel notifications, abuse detection, policy experiments, or high-volume replay and recovery.

2.2.1 Synthetic Population Design and Bias Avoidance

To prevent simulator bias (where the generator guarantees the intervention policy wins), the simulator strictly isolates baseline behavior from intervention pathing:

- **Independent Learner Personas:** Four fixed behavioral archetypes (consistent, binge-then-lapse, weekend-only, at-risk) are defined with independently sampled parameters (session frequency, mastery growth, baseline dropout probability) before any policy selection.
- **Baseline Behavior Generation:** The baseline activity stream is generated first from a fixed random seed with zero knowledge of notifications.
- **Bounded Perturbation:** Notifications apply a capped, independently sampled effect ($\leq \pm 3\text{--}5\%$) on next-day return probability rather than a hardcoded outcome.

- **Common Random Numbers:** Treatment and control arms share the exact same underlying persona population and baseline draws.
- **Sensitivity Sweeps:** Policy decisions are tested across parameter ranges; results that flip under small adjustments are marked inconclusive.
- **Reproducibility & Epistemic Limit:** Parameters and seeds are versioned with the event log for byte-for-byte re-runs. Simulations validate system mechanisms rather than real-world educational causality.

3 Canonical Event Model

The system uses a **replayable event-sourced model** in which gamification state, including points totals, streaks, quest progress, and leaderboard rank, is represented as a projection rebuilt by folding events in order. The gamification service does not own the truth about learning activity, identity, or cohort membership; these are provided by upstream bounded contexts.

```

1 {
2   "$schema": "http://json-schema.org/draft-07/schema#",
3   "title": "CanonicalEventEnvelope",
4   "type": "object",
5   "required": [
6     "event_id",
7     "event_type",
8     "schema_version",
9     "occurred_at",
10    "ingested_at",
11    "subject_id",
12    "producer",
13    "correlation_id",
14    "dedup_key",
15    "payload",
16    "provenance"
17  ],
18  "properties": {
19    "event_id": {
20      "type": "string",
21      "format": "uuid",
22      "description": "ULID/UUIDv7 time-sortable event identifier"
23    },
24    "event_type": {
25      "type": "string",
26      "pattern": "^[a-z]+(\\. [a-z_]+)+$",
27      "examples": ["learning.lesson_completed", "gamification.
28        xp_awarded"]
29    },
30    "schema_version": {
31      "type": "integer",

```

```

32     "description": "Versions the payload shape independently of
        event_type"
33 },
34 "occurred_at": {
35     "type": "string",
36     "format": "date-time",
37     "description": "Timestamp when the event occurred on client
        /source"
38 },
39 "ingested_at": {
40     "type": "string",
41     "format": "date-time",
42     "description": "Timestamp when the message was received by
        Kafka broker"
43 },
44 "subject_id": {
45     "type": "string",
46     "description": "Learner ID, used as the Kafka partition key
        "
47 },
48 "producer": {
49     "type": "string",
50     "examples": ["learning-svc", "assessment-svc", "
        gamification-engine"]
51 },
52 "correlation_id": {
53     "type": "string",
54     "description": "Transaction ID shared across causally
        linked events"
55 },
56 "causation_id": {
57     "type": ["string", "null"],
58     "description": "event_id of the immediate trigger event;
        null for root actions"
59 },
60 "dedup_key": {
61     "type": "string",
62     "description": "Business key for idempotency (e.g., source
        event_id + rule_version)"
63 },
64 "payload": {
65     "type": "object",
66     "description": "Event-specific data matching schema_version
        "
67 },
68 "provenance": {
69     "type": "object",
70     "required": ["source"],
71     "properties": {
72         "source": { "type": "string", "enum": ["client", "
            server_authoritative"] },

```

```

73     "client_app_version": { "type": "string" }
74   }
75 }
76 },
77 "additionalProperties": false
78 }

```

3.1 Shared Event Structure

Every event, whether source or derived, uses the same foundational structure.

1. Global record identifier

- Globally unique, time-sortable identifier.
- Serves as the primary deduplication mechanism at the transport level.

2. Event classification

- Namespaced and versioned category.
- Used by consumers to appropriately route or reject the event.

3. Structure version

- Independent version indicator for the detailed information inside the event classification.
- Allows modifications to the event structure without renaming the overall event classification.

4. Real-world occurrence time

- Records when the real-world action happened according to the producer.
- Used by business logic such as streaks, quiet hours, and timing checks.

5. System ingestion time

- Records when the event entered the processing stream.
- Used for late-arrival detection and operational monitoring.
- Not used for business logic.

6. Subject identifier

- Identifies the learner who is the subject of the event for learner-scoped events.
- Also acts as the natural partition mechanism.

7. Emitting authority

- Identifies the logical service producing the event.
- Used to enforce emission permissions.

8. Business transaction identifier

- Identifies the end-to-end business episode.
- Events causally triggered by the same root action share this identifier.
- Supports tracing and abuse/replay checks.

9. **Direct cause identifier**

- Identifies the specific preceding event that directly caused the current event.
- Allows reconstruction of the exact causal chain.

10. **Business deduplication mechanism**

- Provides a stable deduplication mechanism derived from business identity rather than transport identity.
- Protects against duplicate processing when the global record identifier is re-generated by a retried producer.

11. **Action-specific details**

- Contains the information specific to the particular event type.

12. **Integrity metadata**

- Carries information about how the event was collected.
- Examples include client application or server-authoritative.
- This metadata should not be confused with the business meaning of the event.

13. **Architectural constraints**

- Do not include organizational tenant identifiers unless the infrastructure is actually multi-tenant.
- Do not use generic, schemaless metadata containers.
- Extensibility should be handled through a new structure version.
- Real-world occurrence time and system ingestion time must remain separate because conflating them can make replay non-deterministic.

3.2 **Source vs. Derived Events**

The model distinguishes between **source activity events** and **derived events**.

3.2.1 **Source activity event**

A source activity event represents an action performed by a learner or an upstream system-of-record. Gamification treats the event as an input. These events are owned and produced by domains outside gamification, such as Learning, Assessment, and Enrollment.

3.2.2 Derived event

A derived event represents a conclusion or state change made by the gamification/engagement domain after processing source events. These events are owned and produced by gamification-domain services.

This distinction is important because only derived events are allowed to mutate gamification projections, while source events represent external facts.

3.2.3 Per-Event Specification

1. Lesson Completion

- Represents the learner finishing a learning unit according to the Learning domain's completion criteria. The event identifies the learner, completed learning unit, associated course/context, and occurrence time, with optional duration, methodology, and device information.
- The event is treated as a source activity event and provides the learning activity from which gamification consequences such as reward points and streak updates may be derived.

2. Checkpoint Attempt

- Represents a learner submitting an answer or attempt at an assessment, regardless of the outcome. The event identifies the learner, assessment, parent unit, attempt sequence, and occurrence time.
- The attempt sequence is assigned authoritatively by the server so that the attempt can be uniquely identified and used as an input to later assessment and gamification processing.

3. Checkpoint Completion Outcome

- Represents the Assessment service concluding that a specific attempt passed or failed according to its scoring rules. The event identifies the learner, assessment, attempt evaluation, outcome result, achieved score, and occurrence time.
- The outcome provides the assessment result that can be used by gamification rules, with manual corrections represented through a new event with a revision indicator.

4. Reward Points Award – `xp_awarded`

- Represents the gamification engine deciding to grant or deduct points as a consequence of source events. The event identifies the learner, amount, conceptual reason, rule version, triggering source event ID, and occurrence time.
- The rule version and triggering source event allow the reward decision to be reproduced deterministically during replay and prevent the same logical reward from being awarded multiple times.

5. Streak Update

- Represents the recomputation of a learner’s activity streak after crossing a boundary, such as extending, resetting, or freezing the streak. The event identifies the learner, streak category, new and previous totals, logical calendar date, and rule version.
- The event records the resulting streak state and supports recalculation when late-arriving events affect historical activity.

6. Quest Assignment

- Represents the system determining that a learner is enrolled in a specific challenge or quest. The event identifies the learner, quest definition/version, assignment time, and expiration.
- It records the assignment of the quest so that quest progress can subsequently be evaluated against the assigned challenge.

7. Quest Completion

- Represents the system concluding that all completion criteria for an assigned quest instance have been met. The event identifies the learner, quest instance, completion time, and rule version.
- It records the completed quest state and ensures that a quest instance is completed only once during processing and replay.

8. Cohort Assignment

- Represents an upstream context placing a learner into a specific group, which scopes leaderboard participation. The event is treated as a source activity event owned by external contexts. The event identifies the learner, cohort, effective start time, and occurrence time.
- It provides the cohort information required to determine which group the learner participates in for cohort-based leaderboard activity.

9. Notification Candidate Generation

- Represents the Engagement engine proposing a nudge based on an evaluation policy. The event identifies the learner, proposal instance, intervention type, policy/version, and generation time.
- It records a proposed intervention rather than an actual delivery, allowing the notification system to subsequently determine whether and how the proposal should be delivered.

10. Notification Delivery

- Represents the dispatcher successfully delivering a message corresponding to an approved proposal. The event identifies the learner, delivery record, proposal ID, channel, and send time.
- It records successful communication delivery and is not used to reconstruct gamification state during replay.

11. Notification Failure

- Represents the dispatcher attempting to deliver a message but failing, such as because of opt-out or a provider error. The event identifies the learner, delivery record, proposal ID, failure reason, and failure time.
- It records the failed delivery so that notification outcomes can be distinguished from successful deliveries and handled appropriately during processing.

12. Experiment Assignment

- Represents the system deterministically bucketing a learner into a test variant. The event identifies the learner, experiment, variant, assignment unit, and algorithm version.
- The assignment is a computed pure function, with the event serving as a cached record of the resulting assignment.

13. Experiment Exposure

- Represents the learner actually encountering the experiment’s treatment surface. The event identifies the learner, experiment, variant, exposure time, and surface area.
- It distinguishes actual exposure from experiment assignment and provides the information needed to determine whether the learner encountered the treatment.
- To support counterfactual evaluation and prevent bias from early exploration, the event payload explicitly logs the probability assigned by the policy to the selected variant.

14. Abuse Signal Detection

- Represents Trust systems concluding that a pattern of activity is suspicious. The event identifies the learner, signal instance, abuse type, severity, and evidence event IDs.
- It records the suspicious activity identified by the trust/abuse layer without treating the underlying learner activity as inherently trustworthy; reversals are represented through explicit, separate events.

15. Notification Exposure

- Represents the learner opening or viewing a delivered notification. The event identifies the learner, proposal ID, delivery ID, timestamp, and action type (`push_opened` or `in_app_view`).
- It distinguishes sending a notification from actual user exposure, helping track notification fatigue and experiment results without affecting core gamification state during replay.

16. Badge Issuance

- Represents the gamification engine concluding that a learner met the criteria for a milestone achievement. Identifies the learner, badge type/version (`ex: week_warrior_v1`), triggering condition, occurrence time, and triggering event ID.

- Derived event, issued exactly once per learner per badge type. Idempotency is enforced by a compound check (`learner_id + badge_type`) against the derived state before emission.

4 State-Machine and Replay Rules

To guarantee that replaying identical history produces identical state, the gamification engine is designed as a strict, deterministic state machine. The system isolates the raw event log from the derived state, ensuring that offline events, rule changes, and replays are handled mathematically rather than procedurally.

4.1 The Gamification State Vector

At any given time t , a synthetic learner's state is represented by the vector:

$$S_t = \{XP, StreakCount, Date_{last_active}, Freezes, RuleVersion, BadgesIssued\}$$

4.2 Deterministic Transition Functions

When an event e arrives, the state machine applies a pure transition function

$$f(S_t, e) = S_{t+1}.$$

Before applying state changes, the system enforces deduplication: any incoming event with a **business deduplication identifier** (such as the triggering source event ID) matching a previously processed event within a 7-day rolling window is silently discarded to prevent duplicate processing, bypassing transport-level eventId regeneration.

XP Calculation (with Diminishing Returns) To calculate XP for an event $e_{lesson_completed}$, the system uses an algebraic decay model to prevent reward abuse via repeated lessons. Let E be the base XP defined by the active rule configuration, N be the number of prior completions for this specific lesson, and K be the algebraic decay penalty.

- First attempt ($N = 0$): $XP_{awarded} = E$
- Subsequent attempts ($N \geq 1$): $XP_{awarded} = \max(0, 0.5E - (K(N - 1)))$
- Total Update: $XP_{new} = XP_{current} + XP_{awarded}$

Streak Calculation (Timezone-Aware) Streak boundaries are strictly calculated using the user's localized IANA timezone, independent of server processing time. Let d represent the difference in calendar days:

$$\Delta d = Date_{current\ event} - Date_{last\ active}.$$

- If $d = 0$: $Streak_{new} = Streak_{current}$ (Same day, no change).
- If $d = 1$: $Streak_{new} = Streak_{current} + 1$ (Consecutive day extension).
- If $d > 1$ and $Freezes \geq (d - 1)$: $Streak_{new} = Streak_{current} + 1$, and $Freezes_{new} = Freezes - (d - 1)$ (Streak protected by freeze inventory).
- If $d > 1$ and $Freezes < (d - 1)$: $Streak_{new} = 1$ (Streak broken).

Quest / Remediation State Define a rule where if $Streak_{new} = 1$ (a break occurs), the state machine emits a **Quest Assignment** event scoped to a remediation challenge. If the user completes this specific quest within 24 hours (generating a Quest Completion event), it triggers a **Streak Update** transition to restore the streak.

Badge Issuance

Badge issuance is evaluated as a pure deterministic predicate within $f(S_t, e)$ following state vector updates. For any badge rule R_b mapped to a unique badge identifier b_{id} :

- **Evaluation Condition:** Evaluated whenever an incoming source or derived event e satisfies the rule predicate $P(S_t, e) = \text{true}$.
- **Idempotency Guard:** The state machine checks set membership $b_{id} \in BadgesIssued$.
- **Transition Logic:** If $b_{id} \notin BadgesIssued$, the state machine emits a derived `badge_awarded` event and updates the state vector:

$$BadgesIssued_{new} = BadgesIssued \cup \{b_{id}\}$$

If $b_{id} \in BadgesIssued$, the predicate resolves to a no-op; no event is emitted and *BadgesIssued* remains unchanged.

- **Replay Invariant:** Because *BadgesIssued* is modeled as a set within S_t , subsequent resets, streak breaks, or event-log replays guarantee that b_{id} is awarded exactly once per learner regardless of how many times $P(S_t, e)$ evaluates to true over time.

Example: 7-Day Streak Milestone

1. **Rule Predicate:** $P(S_t, e)$ is true when $StreakCount \geq 7$ upon processing a `streak_updated` event.
2. **Badge Identifier:** `week_warrior_v1` (or `streak_7day_v1`).
3. **Execution:** When a learner hits $StreakCount = 7$, $P(S_t, e)$ evaluates to true. If `week_warrior_v1` $\notin BadgesIssued$, the state machine emits `badge_awarded` and adds `week_warrior_v1` to *BadgesIssued*. If the learner's streak breaks to 0 and reaches 7 again weeks later, the idempotency guard blocks a duplicate event emission.

4.3 Formal Replay and Late-Event Rules

To ensure auditability and consistent rule-version behavior, the system enforces the following protocols:

- **Rule Versioning and Strict Historical Provenance:** When historical state is replayed, the system does not apply the *current* global rules to past events. Instead, upon initial processing, the state machine stamps the active `rule_version` into the payload of the derived `xp_awarded` event. Replays blindly trust these historically stamped derivations, ensuring retroactive rule changes never inflate or deflate past state.

- **The 72-Hour Late-Event Resolution:** Valid offline events must be integrated into the state without silently corrupting historical calculations. If an event arrives where $eventTime < Date_{last\ active}$, and the delay is 72 hours, the system initiates a micro-replay. It drops the current read-model state for that user, retrieves all events from the delayed eventTime forward, and sequentially reapplies the transition functions in memory to construct the mathematically correct present state. Events arriving beyond the 72-hour grace period are rejected to cap computational overhead.

4.4 Cohort and Leaderboard Rules

- **Cohort Boundaries & Stability:** Cohorts are bounded between **500 (minimum)** and **5,000 (maximum)** active learners to ensure rank competition remains meaningful. Assignments reshuffle weekly alongside leaderboard resets. If a learner generates no activity for 30 consecutive days, they are moved to an inactive pool and dropped from active cohort calculations without altering their historical XP or streak state. Mid-cycle dropouts are backfilled from the waitlist/inactive pool to prevent rank churn.
- **Leaderboard Consistency & Tie-Breaking:** Leaderboard rankings operate under an **eventual consistency model** backed by Redis Sorted Sets (p99 latency < 2 s from event ingestion to visibility). Per-learner updates are strictly causally consistent, ensuring a user never sees their own rank move backward relative to their own actions. Ties in scores are broken deterministically by the occurrence timestamp of the event that first reached the score (*earlier event wins*), with ties further resolved by `subject_id`.
- **Reconciliation Process:** A scheduled daily job (and post-incident trigger) audits Redis sorted set scores against the Postgres read projections. Corrections are executed using differential updates rather than flushing state, ensuring recovery from partial failures without clearing real-time ranks.
- **Cohorting Strategies**
 - **Strategy 1: Activity-Volume Cohorts:** Learners are bucketed by recent weekly lesson volume, reshuffled weekly, and bounded to 500–5,000 active members with a 30-day inactivity drop-off.
 - **Strategy 2: Course/Module Cohorts:** Learners are bucketed by their active learning unit via `cohort_assigned` source events. Re-evaluated upon module transition; cohorts under 500 are merged with adjacent modules to preserve competition density across content difficulty tiers.
 - **Exclusion of Sensitive Attributes:** Cohort formation relies strictly on operational activity and enrollment inputs. Demographic data, national origins, or sensitive profile attributes are explicitly excluded from cohorting algorithms to guarantee fairness.

4.5 In-App Delivery and Notification Preferences

To support multiple delivery methods without adding unreliable external dependencies, the notification engine routes alerts through two distinct channels: external **push** notifications (which go through retry queues and dead-letter queues) and **in-app** messages. Unlike push alerts, in-app messages bypass external push networks and are written directly to a database table for the learner’s inbox. Delivery is logged immediately upon database insertion, while user viewing is recorded separately through exposure events.

Every notification candidate carries a time-to-live setting, typically expiring at localized midnight. If a time-sensitive alert is delayed past its expiration window because of quiet hours or retry delays, the engine drops the message and logs a failure event indicating that the notification has become stale rather than sending an outdated nudge.

Learner preferences are tracked per channel and per message category, such as push notifications for streak-risk alerts or in-app leaderboard updates, rather than relying on a single global on/off toggle. Essential operational alerts and in-app messages default to opted-in, while non-essential push notifications default to opted-out. Before generating a notification proposal, the system verifies the learner’s consent for that specific channel and category combination; if opted out, candidate generation is blocked and the failure is logged.

5 Metrics Framework

This framework dictates how the experimentation engine evaluates cohort interventions. It establishes distinct boundaries between actual learning progress, superficial engagement, and platform abuse.

5.1 Primary Learning Outcome

- **First-Attempt Mastery Accuracy:** The core objective is educational progression, not click volume. This metric calculates the average accuracy score across all Checkpoint Attempt events exclusively where the attempt sequence is exactly 1.

5.2 Long-Term Retention & Spaced Repetition

Instead of measuring simple application logins, retention is evaluated through the completion and accuracy of **Spaced Repetition Review Lessons**.

- **Milestone Reviews:** Scheduled systematically at Day 7, Day 15, and Day 30 post-cohort assignment.
- **Intermittent Reviews:** Triggered dynamically after a defined volume of lessons.
 - *Measurement:* Tracking the accuracy on these specific review channels serves a dual purpose: measuring continued platform engagement (retention) while simultaneously gauging the decay of the learner’s mastery over time.
- **Remediation Conversion Rate:** The percentage of lapsed users who successfully complete a remediation quest and generate a valid learning event the following day.

5.3 Guardrails (Experiment Invalidation Criteria)

Experiments that improve engagement at the cost of learner well-being or system integrity are automatically marked as failed.

- **The Notification Fatigue & Escalation Limit:** Enforces a hard cap of 5 intervention push notifications per 24 hours per learner, strictly bounded by localized quiet hours. To support habit formation, “streak at risk” notification intervals decrease exponentially (firing more rapidly) leading up to localized midnight, provided the 5-message daily cap is not breached and localized quiet hours are strictly respected (quiet hours take absolute precedence, truncating any escalating schedule upon entry).
- **The Gaming Threshold:** If $> 15\%$ of a cohort’s total weekly XP generation originates from decayed (repeated) lessons, the specific intervention policy is flagged for reward abuse and disqualified.
- Quiet hours are strictly defined as **10:00 PM to 8:00 AM in the learner’s local timezone**. During this window, all push notification deliveries are blocked regardless of priority or experiment policy.

5.4 Controlled Experiment Specification

Formal Definition (Experiment System Contract)

Every controlled experiment deployed within the platform is formally defined as a tuple:

$$E = \langle exp_id, V, A, M_{\text{primary}}, G, C \rangle$$

where:

- **Experiment Identifier (exp_id):** A unique, immutable string key that scopes state transitions, variant hashing, and exposure logging.
- **Variant Set (V):** A set of mutually exclusive treatment policies

$$V = \{v_0, v_1, \dots, v_k\},$$

where v_0 is strictly designated as the control policy (baseline/no-intervention).

- **Deterministic Assignment Function (A):** A pure function mapping a learner identifier ($subject_id$) and exp_id to a variant using uniform hash bucketing:

$$A(subject_id, exp_id) = H(subject_id \parallel exp_id) \bmod 100$$

where H is a versioned hash algorithm (e.g., MurmurHash3 / Hash Algorithm v2).

- **Primary Metric (M_{primary}):** The core educational or behavioral outcome measure used to evaluate policy efficacy.
- **Guardrails (G):** A set of systemic constraints (e.g., daily notification caps, fatigue thresholds, decayed-XP caps) that, if violated by any treatment variant, automatically invalidate the experiment and halt policy execution.

- **Contamination & SRM Checks (C):** Automated statistical controls, including a chi-square (χ^2) goodness-of-fit test on assignment distribution, to catch Sample-Ratio Mismatch (SRM).

Implementation Example (`exp_007`)

- **Experiment Identifier (exp_id):** "exp_007" (Streak-Risk Reminder Policy vs. Control).
- **Variants (V):**
 - **variant_a** (Treatment): Fires an automated, exponential-interval push reminder during the 4 hours preceding localized midnight when a learner's streak is at risk.
 - **variant_b** (Control): Suppresses streak-risk interventions; observes baseline learner behavior.
- **Assignment & Exclusions:** 50/50 split via $H(subject_id \parallel "exp_007") \bmod 100$. Excludes learners with platform tenure < 3 days or those who opted out of the push $\times$ streak_risk consent pair.
- **Primary Metric (M_{primary}):** First-Attempt Mastery Accuracy over a 14-day window (with Remediation Conversion Rate as the short-horizon operational signal).
- **Guardrails (G):** The experiment is automatically marked as invalidated if **variant_a** causes push notifications to exceed 5 messages per 24 hours (§5.3) or drives cohort decayed-XP share above 15% (§5.3).
- **SRM Safeguard (C):** A daily pipeline job runs a χ^2 test comparing actual assignment ratios against the expected 50/50 split ($p < 0.001$ triggers an SRM alert).

5.5 Bandit Policy Allocation

We will use a contextual multi-armed bandit for notification policy selection, rather than fixed A/B allocation, but with propensity logging as a hard requirement from day one. Our literature review (paper 4) surfaced the central risk directly: a bandit can converge on an arm before there's statistically significant evidence it's actually better, and it optimizes an easy-to-measure proxy (email/push opens) rather than the outcome we actually care about (learning).

We mitigate this by:

1. Logging the propensity score: the probability the policy assigned to the arm it picked, on every *Experiment Assignment* event, which lets us later re-weight results with inverse-propensity scoring instead of trusting raw bandit outcomes at face value; and
2. Capping exploitation with a minimum exploration floor (e.g., ϵ -greedy with $\epsilon \geq 0.1$) for the first two weeks of any experiment, so no arm is starved of data before we have enough exposures to trust the comparison.

We are not using a bandit for core reward-rule selection (XP amounts, decay rates), those stay fixed and rule-versioned.

5.6 System-Performance & Reliability Metrics

This framework maps directly to the required milestones (M1–M8) to prove the system handles scale, recovery, and strict consistency.

- **Peak-Load Ingestion & Idempotency (M1, M8):** The P99 latency measured from event generation to canonical storage under a synthetic high-volume workload, including the exact computational overhead of verifying business transaction identifier and dropping duplicate events using their business deduplication identifiers.
- **State Rebuild & Recovery Time (M2, M8):** The exact time required to sequentially read the append-only event log and completely reconstruct the deterministic gamification state (XP, streaks, badges) for the synthetic population following a simulated database wipe.
- **Leaderboard Consistency & Latency (M3, M8):** The measurable update and query latency for Redis sorted sets at both the cohort and global levels, tracking the time delta between event ingestion and read-model availability.
- **Notification Reliability & Dead-Letter Routing (M5, M8):** The percentage of failed notification candidates successfully routed to the Dead-Letter Queue (DLQ), alongside the recovery time required to process a backlog of delayed retry events after a simulated external channel failure.
- **Abuse Detection Efficacy (M6):** The latency and success rate of the rules engine in identifying scripted point farming or impossible timing exploits, measured by the volume of canonical events successfully flagged with stored evidence rather than silently dropped.

6 Architecture Alternatives

6.1 Method A: PostgreSQL-Based Event Sourcing & Outbox Pattern

To prevent destructive state mutations and fulfill the requirement to rebuild state from history, this architecture uses PostgreSQL as an append-only event store rather than a traditional CRUD mutating database. It utilizes the Transactional Outbox pattern to queue downstream side-effects.

6.1.1 Components

- **Synthetic Event Generator:** Simulates learner activity and submits REST payloads.
- **Gamification API Layer:** Receives payloads and processes business logic.
- **Relational Database (PostgreSQL):** Acts as the central system of record. It maintains an append-only events table (storing JSONB envelopes) and an outbox table for side-effects. To provide fast reads, it also maintains dual-written projection tables (e.g., `user_state`). All three table updates are wrapped in a single ACID transaction.

- **Outbox Relay:** A background worker or Change Data Capture (CDC) process that continuously polls the outbox table and pushes candidate events to external notification adapters.

6.1.2 Strengths

- **Atomic Consistency:** The database natively prevents race conditions during concurrent updates via ACID transactions, ensuring the event log, state projection, and outbound messages are committed simultaneously.
- **Familiar Tooling:** Developers familiar with SQL can build the initial schema and API layer rapidly without managing distributed infrastructure.

6.1.3 Architectural Mismatches

- **Concurrency Bottlenecks (Row-Level Locking):** Under high synthetic workloads, multiple concurrent events for the same user or shared resources force the database to acquire row-level locks (e.g., `SELECT ... FOR UPDATE`). This rapidly saturates connection pools, leading to latency spikes and query timeouts.
- **Late Event Brittleness:** Integrating an offline event that arrives 48 hours late requires brittle, custom SQL logic. The system must insert the delayed event, and the application must execute a retrospective script to patch the `user_state` projection table and manually roll back any incorrect downstream outbox triggers.
- **Coupled Infrastructure:** All domains (gamification, leaderboards, abuse detection) compete for the same database compute resources and poll the same outbox table, creating a monolithic bottleneck.

6.2 Method B: Event Pipeline & CQRS (Kafka/Redpanda + Redis)

This architecture utilizes a distributed message broker as an immutable event bus. It strictly separates the write-path (appending events) from the read-path (querying state projections), achieving native compartmentalization.

6.2.1 Components

- **Synthetic Event Generator:** Publishes versioned canonical events directly to Kafka topics, partitioned by `userId`.
- **Event Broker (Kafka/Redpanda):** A distributed commit log that stores events immutably. Partitioning guarantees strict chronological ordering and sequential processing for every individual learner.
- **Gamification State Machine:** A consumer service that reads the stream, applies pure transition functions, emits derived events (e.g., `xp_awarded`) back to the broker, and updates the primary Postgres profile projection.
- **Specialized Read Models (Redis):** Independent Redis Sorted Sets continuously updated by consumers to provide sub-millisecond, low-latency cohort leaderboards.

- **Domain-Specific Consumers:** Independent microservices that subscribe to the event bus to handle abuse detection logging, multi-channel notifications, and experiment telemetry without blocking the main state machine.

6.2.2 Strengths

- **Compartmentalization of Concerns:** The Kafka log acts as a shared backbone. The Leaderboard service (M3), Abuse Detection engine (M6), and Notification engine (M5) scale independently, consume the same events at their own pace, and handle their own failures without impacting the core XP calculation logic.
- **Lock-Free Concurrency:** Because Kafka partitions ensure all events for a specific `userId` flow into a single partition processed sequentially by a single thread, race conditions are eliminated natively. The system processes massive throughput without the overhead of database row-level locking.
- **Native Replayability and Late-Event Resolution:** Rebuilding state is executed simply by resetting a consumer offset. For late offline events, the state machine executes an in-memory recalculation of the affected time window and appends a newly calculated Streak Update event. Downstream consumers independently react to this correction without requiring complex, cross-table SQL rollbacks.

6.2.3 Architectural Mismatches

- **Eventual Consistency:** Because read-models (Redis/Postgres) update milliseconds after an event is appended to the log, client applications must be designed to tolerate slight read delays immediately following an action.
- **Operational Overhead:** Deploying, tuning, and maintaining a message broker cluster and managing consumer group offsets introduces a steeper learning curve and higher infrastructure complexity than a single PostgreSQL instance.

7 Architectural Selection

We proceed with **Method B: Event Pipeline & CQRS (Kafka/Redpanda + Redis)**, selected primarily for its fit with our 8-week timeline and 3-person team capacity. The core deliverable of this project is provable replay and deterministic state reconstruction (M2, M8), which Method B fulfills directly: resetting a consumer offset and re-folding events serves as the replay mechanism itself rather than bespoke logic we must hand-build. Under Method A, replay and late-event handling require custom SQL scripts that patch projection tables by hand—an error-prone overhead that would consume disproportionate engineering time while failing to generalize across our parallel abuse, leaderboard, and notification consumers.

Method B’s secondary advantage directly mirrors our milestone delivery structure. M3 (leaderboards), M5 (notifications), and M6 (abuse detection) represent independent read paths off the same event log. Implementing them as isolated Kafka consumers enables parallel development, testing, and debugging across our 3-person team without serializing work on a shared database schema or lock bottleneck. We explicitly accept the operational overhead and learning curve of running an event broker over a single

Postgres instance; Method B directly exercises the event-sourcing and replay invariants the rubric evaluates, whereas Method A would divert limited engineering weeks toward writing ad-hoc scaffolding around a database that does not natively support them.

Rule-Based Abuse Detection Rationale

We are choosing rule-based, retroactive abuse detection over a real-time ML anomaly-detection model, for both architectural and feasibility reasons. Architecturally, abuse detection is designed to operate asynchronously on the event stream, so it does not interfere with real-time event ingestion. An ML model would add latency and a training/retraining lifecycle that isn't compatible with the deterministic-replay requirement. Feasibility-wise, an 8-week team building its first event-sourced system does not have the labeled abuse data or the runway to validate a model's false-positive rate. but a rule set is auditable, testable, and directly traceable to the evidence-event-ID requirement. We treat ML-based anomaly detection as a stated future extension, not a W1-W8 deliverable.

8 Treaceability & Acceptance tests

8.1 Milestone Definitions

Milestone	Definition
M1	Peak-load ingestion & idempotency
M2	State rebuild & recovery
M3	Leaderboard consistency & latency
M4	Experimentation Engine - deterministic assignment, exposure tracking, propensity logging
M5	Notification reliability & DLQ routing
M6	Abuse detection efficacy
M7	Integration & Recovery Drill - full cross-consumer replay after simulated failure
M8	Full event-log replay / end-to-end rebuild

Table 1: System Milestones

8.2 Traceability Matrix

Milestone	Acceptance Test(s)	Component
M1	AT-1	Ingestion API + envelope schema
M2	AT-2, AT-3, AT-4, AT-5, AT-16	State machine + replay
M3	AT-7	Leaderboard service (Redis)
M4	AT-12, AT-13	Experiment/bandit engine
M5	AT-8, AT-9, AT-10, AT-11, AT-14	Notification engine
M6	AT-6	Abuse rule engine
M7	AT-15 (partial)	Cross-consumer integration
M8	AT-2, AT-15	Full rebuild after DB wipe

Table 2: Milestone Acceptance Tests and Components

8.3 Acceptance Tests

1. **Duplicate Event Dedup (M1)**: Given learner L1 submits `lesson.completed` for `l_492` and receives `XP_awarded = E` (first attempt, $N = 0$), when the identical event (same `dedup_key`) is re-ingested within the 7-day window, then it is silently discarded and `XP_total` remains unchanged.
2. **Deterministic Replay Checksum (M2, M8)**: Given a fixed sequence of 100 events for learner L1, when the event log is replayed twice from offset 0 via $f(S_t, e) = S_{t+1}$, then the resulting state vectors $\{\text{XP}, \text{StreakCount}, \text{Freezes}, \text{RuleVersion}\}$ are bit-identical both times.
3. **Out-of-Order Within 72h (M2)**: Given learner L1's last processed event has `occurred_at = 2026-09-22`, when an event with `occurred_at = 2026-09-21` arrives at `ingested_at = 2026-09-24` (within 72h), then a micro-replay is triggered from 2026-09-21 forward.
4. **Late Event Streak Recalc (M2)**: Given `StreakCount = 5`, `Freezes = 1`, last active 2026-09-20, when a valid event with `occurred_at = 2026-09-22` ($\Delta d = 2$) arrives within 72h, then $\text{Freezes} \geq (\Delta d - 1) = 1$ holds, so `StreakCount = 6` and `Freezes = 0`.
5. **Historical Rule Versioning (M2)**: Given `xp_awarded` was stamped with `rule_version = v1` when $E = 100$, when the global rule config changes to $E = 150$ and replay occurs, then the historical event still yields `XP_awarded = 100`, not 150.
6. **Point Farming / Decay Cap (M6)**: Given $E = 100$, $K = 20$, learner completes lesson `l_492` for the 3rd time ($N = 2$), when XP is calculated, then

$$\text{XP_awarded} = \max(0, 0.5(100) - 20(1)) = 30$$

and if this pattern pushes a cohort's decayed-XP share above 15% of weekly total, the policy is flagged.

7. **Leaderboard Reconciliation (M3)**: Given Redis and Postgres diverge on L1's score by a dropped event, when the daily reconciliation job runs, then it applies a differential ZADD correction (not a full flush) and resolves any tie using `occurred_at` of the first event reaching that score, then `subject_id`.
8. **Quiet-Hour Delay (M5)**: Given L1's timezone is `Asia/Kolkata` with quiet hours 22:00–07:00, when a streak-risk candidate generates at 23:10 local, then delivery reschedules to 07:00 next day and counts against that day's 5-message cap.
9. **Duplicate Notification Proposals (M5)**: Given `proposal_id = p_001` was already generated for L1, when the policy attempts to generate a second candidate for the same trigger, then it is deduplicated by `proposal_id` before dispatch.
10. **Retry & DLQ (M5)**: Given a notification delivery attempt fails due to a provider error, when retries are exhausted, then the message routes to the DLQ and is recoverable after the channel failure is resolved.
11. **Opt-Out Enforcement (M5)**: Given L1 has opted out of push, when a candidate is generated for L1, then `Notification Failure` is emitted with `failure_reason = opt_out`, not a delivery attempt.

12. **Deterministic Experiment Assignment (M4):** Given learner L1 and experiment `exp_007` with a fixed hash algorithm `v2`, when assignment runs twice, then L1 is bucketed into the same variant both times, with the propensity score logged on the event.
13. **Assignment vs. Exposure (M4):** Given L1 is assigned to `variant_a` for `exp_007`, when L1 has not yet opened the app, then only **Experiment Assignment** exists. **Experiment Exposure** is emitted only once L1 actually views the treatment surface.
14. **Notification Fatigue Guardrail (M5):** Given L1 has already received 5 push notifications in the current 24h window, when a 6th candidate is generated, then it is blocked and the triggering experiment variant is flagged for guardrail violation.
15. **Full State Rebuild (M2, M7, M8):** Given a simulated database wipe for the full synthetic population, when the event log is replayed from offset 0, then all learners' `{XP, StreakCount, Freezes}` match their pre-wipe values exactly, and leaderboard/cohort projections rebuild to the same ranks.
16. **Badge Idempotency (M2):** Given learner L1 meets the condition for badge b_{id} (e.g., `week_warrior_v1`) and `badge_awarded` is emitted, when the triggering state condition breaks and is rebuilt to meet the criteria again, then no second `badge_awarded` event for b_{id} is emitted and *BadgesIssued* contains exactly one instance of b_{id} .